

Day 9 : Animation And Transition

1. Introduction to Animation

Animation is the process of changing a widget's properties over time to create movement or visual effects.

Examples:

- Button enlarges when clicked
- Screen fades in
- Image moves from one page to another
- Loading spinner rotates

2. Why Do We use Animation?

- Makes UI attractive
- Improves user experience
- Provides visual feedback
- Makes navigation smoother
- Makes apps look professional

3. Animation Types in flutter

Flutter provides two main types of animations:

A. Implicit Animations

Flutter automatically handles the animation.

You only change widget properties and Flutter animates the changes.

Advantages

- Easy to implement
- Less code
- Good for beginners

Examples

- AnimatedContainer
- AnimatedOpacity
- AnimatedPositioned
- AnimatedCrossFade
- TweenAnimationBuilder

B. Explicit Animations

Developer controls every animation step.

Uses:

- AnimationController
- Tween
- Animation
- AnimatedBuilder

Advantages

- More control
- Complex animations possible
- Custom effects

4. AnimatedContainer

Animated version of Container.

Used to animate:

- Width
- Height
- Color
- Padding
- Margin
- Border Radius

```
AnimatedContainer(  
  duration: Duration(seconds: 1),  
  width: isExpanded ? 200 : 100,  
  height: isExpanded ? 200 : 100,  
  color: isExpanded ? Colors.blue : Colors.red,  
)
```

Working

1. User clicks button.
2. State changes.
3. Container properties change.
4. Flutter animates old values to new values.

Use Cases

- Expand cards
- Animated buttons

5. AnimatedOpacity

Animates transparency.

Opacity values:

- 0 = Invisible
- 1 = Fully Visible

```
AnimatedOpacity(  
  opacity: visible ? 1.0 : 0.0,  
  duration: Duration(seconds: 1),  
  child: FlutterLogo(),  
)
```

6. AnimatedPositioned

- Animates widget position.
- Must be used inside Stack.

```
Stack(  
  children: [  
    AnimatedPositioned(  
      duration: Duration(seconds: 1),  
      left: move ? 200 : 0,  
      child: Icon(Icons.star),  
    )  
  ],  
)
```

7. AnimatedCrossFade

Switches smoothly between two widgets.

Example

```
AnimatedCrossFade(  
  firstChild: Text("Login"),  
  secondChild: Text("Welcome"),  
  crossFadeState:  
    showFirst  
    ? CrossFadeState.showFirst  
    : CrossFadeState.showSecond,  
  duration: Duration(seconds: 1),  
)
```

8. TweenAnimationBuilder

Tween = Transition Between Values

Used to animate any value.

```
TweenAnimationBuilder(  
  tween: Tween(begin: 0.0, end: 1.0),  
  duration: Duration(seconds: 2),  
  builder: (context, value, child) {  
    return Opacity(  
      opacity: value,  
      child: child,  
    );  
  },  
  child: Text("Flutter"),  
)
```

9. Returning Data Backward

Animation Curves

Curves.easeIn

Starts slow and ends fast.

Curves.easeOut

Starts fast and ends slow.

Curves.bounceIn

Creates bouncing effect.

Custom Curves

Create your own animation behavior.

One Line: Curves control animation speed and movement style.

10. Explicit Animations

When greater control is needed.

Components:

1. AnimationController
2. Tween
3. Animation
4. Listener

A. AnimationController

Heart of explicit animations.

Controls:

- Start
- Stop
- Reverse
- Repeat

```
late AnimationController controller;  
  
controller = AnimationController(  
  vsync: this,  
  duration: Duration(seconds: 2),  
);
```

B. Tween

Defines animation range

```
Tween<double>(  
  begin: 0,  
  end: 100,  
);
```

C. Animation Listener

Updates UI continuously.

```
animation.addListener(() {  
  setState(() {});  
});
```

D. Dispose Controller

Important for memory management.

```
@override  
void dispose() {  
  controller.dispose();  
  super.dispose();  
}
```

11. Hero Animations

Creates animation between two screens.

Widget appears to "fly" from one page to another.

```
Hero(  
  tag: "profile",  
  child: CircleAvatar(  
    radius: 40,  
  ),  
)
```

```
Hero(  
  tag: "profile",  
  child: CircleAvatar(  
    radius: 120,  
  ),  
)
```

12. Custom Page Transition

```
Navigator.push(  
  context,  
  MaterialPageRoute(  
    builder: (_) => SecondPage(),  
  ),  
);
```

```
PageRouteBuilder(  
  pageBuilder: (_, __, ___) => SecondPage(),  
  transitionsBuilder:  
    (_, animation, __, child) {  
    return FadeTransition(  
      opacity: animation,  
      child: child,  
    );  
  },  
);
```

Types

- Fade
- Slide
- Scale
- Rotation

13. AnimatedBuilder

Used with explicit animations.

Rebuilds only necessary widgets.

```
AnimatedBuilder(  
  animation: controller,  
  builder: (context, child) {  
    return Transform.rotate(  
      angle: controller.value * 6.28,  
      child: child,  
    );  
  },  
  child: Icon(Icons.refresh),  
)
```

Code :

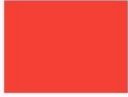
```
import 'package:flutter/material.dart';  
  
Run | Debug | Profile  
void main() {  
  runApp(MaterialApp(  
    debugShowCheckedModeBanner: false,  
    home: AnimationDemo(),  
  )); // MaterialApp  
}  
  
class AnimationDemo extends StatefulWidget {  
  @override  
  State<AnimationDemo> createState() => _AnimationDemoState();  
}  
  
class _AnimationDemoState extends State<AnimationDemo> {  
  bool selected = false;  
  bool visible = true;  
  
  @override  
  Widget build(BuildContext context) {  
    return Scaffold(  
      appBar: AppBar(title: Text("Flutter Animations")),  
      body: Center(  
        child: Column(  
          mainAxisAlignment: MainAxisAlignment.center,  
          children: [  
            AnimatedContainer(  
              duration: Duration(seconds: 1),  
              width: selected ? 200 : 100,  
              height: selected ? 200 : 100,  
              color: selected ? Colors.blue : Colors.red,  
            ), // AnimatedContainer  
  
            SizedBox(height: 20),
```

```
            ElevatedButton(  
              onPressed: () {  
                setState(() {  
                  selected = !selected;  
                });  
              },  
              child: Text("Animate Container"),  
            ), // ElevatedButton  
  
            SizedBox(height: 20),  
  
            AnimatedOpacity(  
              duration: Duration(seconds: 1),  
              opacity: visible ? 1.0 : 0.0,  
              child: FlutterLogo(size: 80),  
            ), // AnimatedOpacity  
  
            ElevatedButton(  
              onPressed: () {  
                setState(() {  
                  visible = !visible;  
                });  
              },  
              child: Text("Fade Animation"),  
            ), // ElevatedButton  
          ],  
        ), // Column  
      ), // Center  
    ); // Scaffold  
  }  
}
```

3:31

Bluetooth, Wi-Fi, Cellular, and Battery status icons

Flutter Animations



Animate Container



Fade Animation